



Java Assignment: Multi-Movie Cinema Management System

1. Objective

Build a Java application that manages multiple movies in a cinema, using arrays to store movie information and seating arrangements.

The system should allow users to view movies, inspect seat availability, reserve seats, and manage the movies available in the cinema.

You will extend your knowledge of two-dimensional arrays by managing several seating arrangements within one application.

2. The Problem

A cinema has multiple movie screens. Each screen shows a particular movie and has its own seating arrangement.

For this assignment, assume that each movie has its own screen.

The cinema initially shows three movies:

Movie	Rows	Seats per row
The Lion Adventure	4	5
Space Explorers	5	6
Mystery at Midnight	6	4

Each seat can have one of two states:

- - O — Available
- X — Reserved

When the program starts, all seats are available.



Your program must store the movie names and their corresponding seating arrangements.

3. Basic Requirements

Your program must:

- 1. Store the names of the three movies in an array.
- 2. Represent each movie's seating arrangement using a two-dimensional character array.
- 3. Initialize all seats as available (O).
- 4. Display the list of movies.
- 5. Allow the user to select a movie.
- 6. Display the seating arrangement for the selected movie.
- 7. Allow the user to reserve a seat by entering its row number and seat number.
- 8. Update the selected movie's seating arrangement after a successful reservation.
- 9. Display the updated seating arrangement.

Example: Movie Selection

```
None
=====

CINEMA MANAGEMENT

=====

Available Movies

1. The Lion Adventure
2. Space Explorers
3. Mystery at Midnight

Choose a movie: 2
```

Example: Displaying Seats



None

Movie: Space Explorers

1 2 3 4 5 6

Row 1 0 0 0 0 0 0

Row 2 0 0 0 0 0 0

Row 3 0 0 0 0 0 0

Row 4 0 0 0 0 0 0

Row 5 0 0 0 0 0 0

Enter row number: 3

Enter seat number: 4

Seat reserved successfully!

The system must update the seating arrangement belonging to the selected movie, not another movie.

4. Basic Completion Condition

Your program is complete when it can:

- - Store multiple movie names.
- - Maintain a separate seating arrangement for each movie.
- - Display the movie list.
- - Allow a user to select a movie.
- - Display that movie's seats.
- - Reserve a seat and update the correct seating arrangement.

The optional challenges are not required for basic completion.



5. Optional Challenges

Challenge 1: View All Movies

Display all movies in a well-organized list, including their screen number, seating dimensions, and total number of seats.

Example:

None

MOVIES CURRENTLY SHOWING

Screen 1: The Lion Adventure

Rows: 4

Seats per row: 5

Total seats: 20

Screen 2: Space Explorers

Rows: 5

Seats per row: 6

Total seats: 30

Screen 3: Mystery at Midnight

Rows: 6

Seats per row: 4

Total seats: 24

Calculate the total number of seats from the seating dimensions.

Challenge 2: Multiple Reservations

Allow users to reserve several seats for a selected movie without restarting the program.

After every successful reservation, display the updated seating arrangement.



Challenge 3: Prevent Double Booking

Do not allow a user to reserve a seat that is already occupied.

Display a message explaining that the selected seat is unavailable.

Challenge 4: Check a Specific Row or Column

Allow the user to select a movie and then choose whether to inspect a particular row or column.

The user must provide the row or column number.

Display the availability of every seat in that row or column and calculate the number of available seats.

Challenge 5: Display Movie Statistics

For a selected movie, display:

- - Total seats.
- - Reserved seats.
- - Available seats.
- - Percentage of seats reserved.

Also provide an option to display these statistics for every movie.

Challenge 6: Add a Movie

Allow the user to add a new movie to the cinema.

The program should ask for:

- 1. The movie name.
- 2. The number of seating rows.
- 3. The number of seats per row.

Create a seating arrangement of the specified dimensions and initialize every seat as available.

The new movie should appear in the movie list and be available for reservations.

Example:



None

Enter movie name: Kingdom of Tomorrow

Enter number of rows: 5

Enter seats per row: 8

Movie added successfully!

Challenge 7: Choose the Movies at Startup

Instead of always starting with the three predefined movies, allow the user to choose how many movies the cinema will show.

Ask the user for the number of movies, then collect each movie's name, number of rows, and seats per row.

Initialize every movie's seating arrangement automatically.

For example:

None

How many movies will the cinema show? 2

Enter movie 1 name: Ocean World

Enter number of rows: 4

Enter seats per row: 6

Enter movie 2 name: The Last Kingdom

Enter number of rows: 5

Enter seats per row: 8

For this challenge, the program may start with user-provided movies instead of the predefined list.

Challenge 8: Rename a Movie

Allow the user to select an existing movie and change its name.

The seating arrangement and reservations associated with that movie must remain intact.



Challenge 9: Change Seating Dimensions

Allow the user to change the number of rows or seats per row for a selected movie.

Consider how existing reservations should be handled.

For this assignment, you may require the movie to have no reservations before its seating dimensions can be changed. Alternatively, design a clear process for handling existing reservations.

Challenge 10: Remove a Movie

Allow the user to remove a movie from the cinema.

Before removing it, display the movie's details and ask the user to confirm.

If the movie has existing reservations, warn the user and require confirmation before proceeding.

After removal, the remaining movies must still be accessible, and their seating arrangements must remain associated with the correct movies.

Challenge 11: Search for a Movie

Allow the user to search for a movie by name.

Display the movie's details and seating availability if a matching movie is found.

Consider allowing partial-name searches, such as searching for Kingdom to find The Last Kingdom.

Challenge 12: Movie Reservation Receipt

Allow a passenger to reserve a seat using their name.

After a successful reservation, display a receipt containing:

- - Reservation number.
- - Passenger name.
- - Movie title.
- - Screen number.
- - Seat row and number.
- - Date and time of reservation.
- - Reservation status.

Example:



None

=====

CINEMA RESERVATION

=====

Reservation No: CM-0001

Passenger: Ada Okafor

Movie: Ocean World

Screen: 1

Seat: 2B

Date: 03 October 2026

Time: 11:30 AM

Status: CONFIRMED

=====

The date, time, and reservation number are illustrative. Your program must generate the actual date and time and assign a unique reservation number.

Consider how to store passenger and reservation details so that they remain associated with the correct movie and seat.

Challenge 13: Cancel a Reservation

Allow a user to cancel an existing seat reservation.

The selected seat must become available again, and any associated passenger information must be handled appropriately.

If you implemented reservation receipts, consider using the reservation number to identify the reservation being cancelled.

Challenge 14: Cinema Management Menu

Create a menu that brings the application's operations together.

312, Herbert Macaulay Way, Sabo, Yaba.

+234(0) 906 000 8609

info@semicolon.africa

www.semicolon.africa



Example:

```
None
=====

  CINEMA MANAGEMENT SYSTEM

=====

1. View all movies
2. View a movie's seating arrangement
3. Reserve a seat
4. Check row or column availability
5. View movie statistics
6. Add a movie
7. Rename a movie
8. Change seating dimensions
9. Remove a movie
10. Search for a movie
11. Cancel a reservation
12. Exit

Choose an option:
```

Your menu should display the options you have implemented. After an operation is completed, return to the menu until the user chooses to exit.

6. Important Restrictions

- - Use arrays to store movie information.
- - Use two-dimensional arrays to represent individual seating arrangements.
- - Use loops to display movies and seats.



- - Do not create separate variables for every movie or every seat.
- - All reservations must update the correct seating arrangement.
- - Availability statistics must be calculated from the current data.
- - When adding, renaming, or removing movies, ensure that the correct seating arrangement remains associated with each movie.

You may use additional arrays to store information that belongs to each movie.

7. What You Are Practising

This assignment helps you practise:

- - One-dimensional arrays.
- - Two-dimensional arrays.
- - Nested loops.
- - Array indexing.
- - Managing multiple related collections of data.
- - Dynamically creating arrays.
- - Searching and updating array contents.
- - Counting and calculating statistics.
- - Collecting user input.
- - Working with dates and times.
- - Designing menu-driven applications.

8. Submission Requirements

Submit:

1. Your Java source code.
2. A screenshot showing the program running.
3. Examples demonstrating the optional challenges you completed.

Start with the basic requirements before attempting the optional challenges.

312, Herbert Macaulay Way, Sabo, Yaba.

+234(0) 906 000 8609

info@semicolon.africa

www.semicolon.africa



- The goal is to build a working cinema system by extending your understanding of arrays and using them to manage multiple related seating arrangements.